AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_task_8 Version : 1	

HW/SW PROTOTYP AV KOMMUNIKATIONS TYPER MED REAL-TIME KERNEL(RTK) SYSTEM

Darwin Rengifo

2020-07-25

Sammanfattning:
Denna rapport beskriver en förstudie och planering för utvecka kunskap om realtidskärnor med användningen av Sierra Real-Time Kernel(RTK-accelerator), DE10-Lite kortet, brytare för att testas Multi-tasking jobb, tre lysdioderna från kortet som visar Multi-tasking aktivitet och en knappen KEY[0] för att testar interruptus. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1. INLEDNING 2
2. KRAVSPECIFIKATION FRÅN KUND 3
3. VANLIGA STYSMEKANISMER FÖR SAMTIDIGHET 4
3.1 Semafor och flaga 4
3.2 Globala variabler 4
3.3 Sierra IRQ 4
4. SYSTEM ARKITEKTUR FÖR INGÅENDE KOMPONENTER 4
4.1 Systemarkitektur 5
5. MJUKVARAN ARKITEKTUR 7
5.1 Minneskarta 7
5.2 Programarkitektur 7

1. INLEDNING

Denna rapport beskriver tre olika sätt av få kunskap om realtidskärnor och hur programvaran fungerar i ett inbyggt system.
De tre olika sätt är semafor och flagga, globala variabler och den IRQ av Sierra.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört
	Krav	Ja/nej
001	Beskriv i ett kapitel i leveransen: 1) Semafor och flagga 2) Globala variabler 3) Sierra IRQ Kunden ska använda detta som informationsmaterial på mässor.	Ja
002A	Använd semafor för ”printf” från kunduppdrag RTK A (task 7), utan tryck knappar), för att göra en säkrare lösning.	Ja
002B	Använd globala variabler istället för ”printf” från kunduppdrag RTK A, Task_A och Task_C (utan tryck knappar), och Task_B skriver ut resultatet med ”printf”.	Ja
002C	Ta fram en HW/SW-konstruktion som skriver till terminalfönstret med printf då en tryckknapp trycks ner och då den släpps. Tryckknappen kopplas via en interfacekomponent till IRQ- ingången på Sierra. En ”task” väntar på IRQ och skriver ut en informationstext om tryckknappen är nertryckt eller inte. EN extra PIO måste också anslutas så att programmet vet om tryckknappen är nertryckt eller inte. IRQ ska ges på båda flankerna från tryckknappen.	Ja
003	Regler och riktlinjer för VHDL och C ska följas.	Ja
	Leverans	
004	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en rapport med ovanstående krav. Namnet på filen ska vara ”fornamn_efternamn_HWSW_task_8”. Sista leveransdag se kursschema (för VG).	Ja

3. VANLIGA STYSMEKANISMER FÖR SAMTIDIGHET

3.1 Semafor och flaga

Semaforer används när flera processer använder samma resurs, t.ex. när det läser och skriver till ett minne. Detta innebär att designern kan skydda resursen från fel läsning och skrivning med en semafor och därmed se till att bara en process kan skriva eller läsa åt gången. En semafor kan endast användas av en process åt gången. När en process har tagit flaggan, kan ingen annan göra det. Processer tar samma flagga tills den returneras och fortsätter.

Flaggor skiljer sig helt från semaforer eftersom flera olika processer kan använda samma flagga samtidigt. En process kan också vänta på att ett tillstånd inträffar för en eller flera flaggor. Flaggor kan därför användas för att starta eller stoppa flera processer samtidigt och så vidare för att synkronisera flera processer i ett system. Flaggor kan också användas för att kontrollera processer till olika tillstånd.

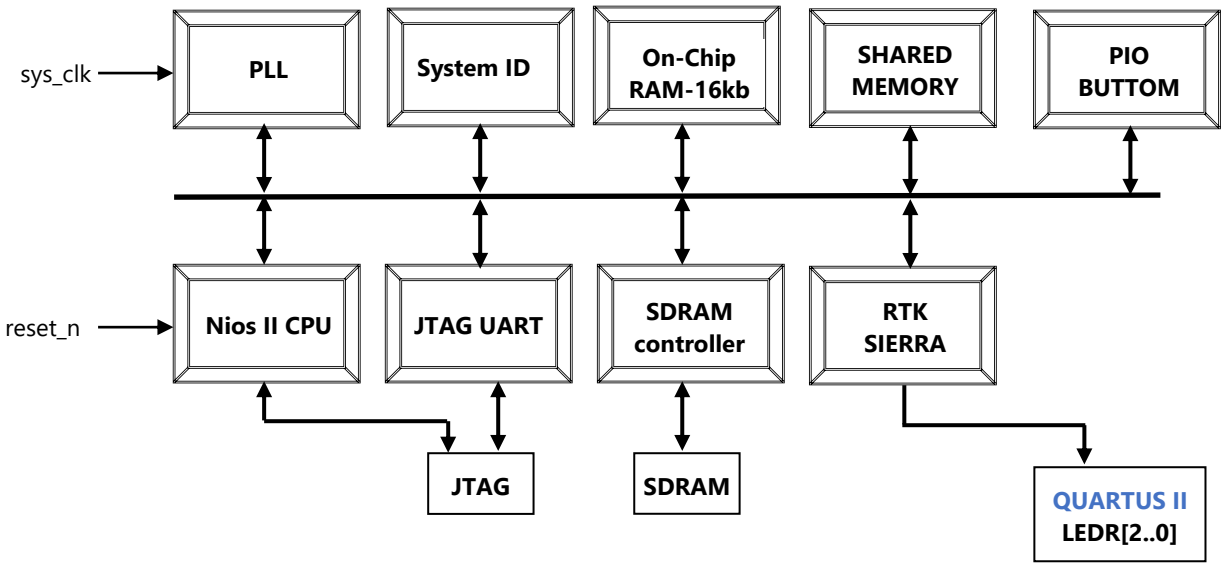
3.2 Globala variabler

Globala variabler kan användas för att överföra och dela information mellan olika processer på ett snabbt och enkelt sätt. En global variabel deklareras utanför huvudfunktionen och eftersom kompilatorn inte krävs. Optimera en global variabel när det sammanställer med "flyktigt" prefix.

3.3 Sierra IRQ

Denna funktion är unik för RTK, den är en IRQ och den är schema som en task. Det betyder att om en task med högre prioritet exekverar och en lägre IRQ begärs kommer det inte att påverka exekveringen för CPU innan IRQ-task har högsta prioritet. När du använder FPGA kan enhetsdrivrutinen implementeras på ett enkelt sätt i HW, med HW-språket.

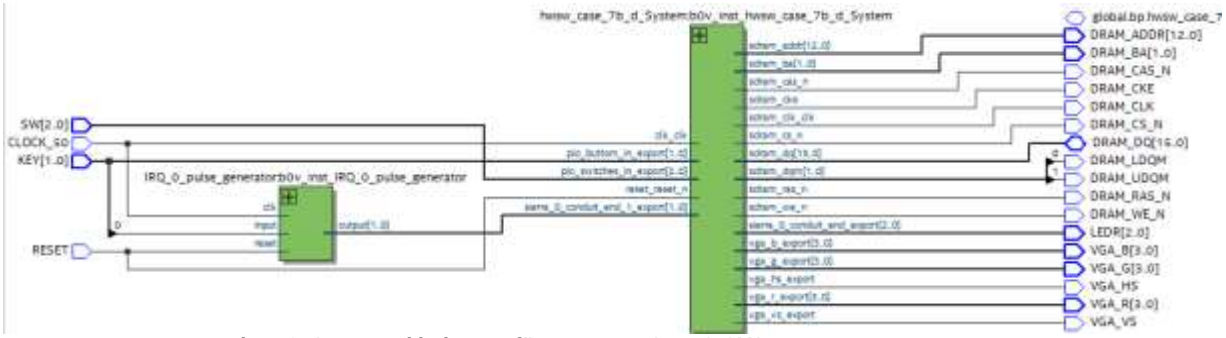
4. SYSTEM ARKITEKTUR FÖR INGÅENDE KOMPONENTER



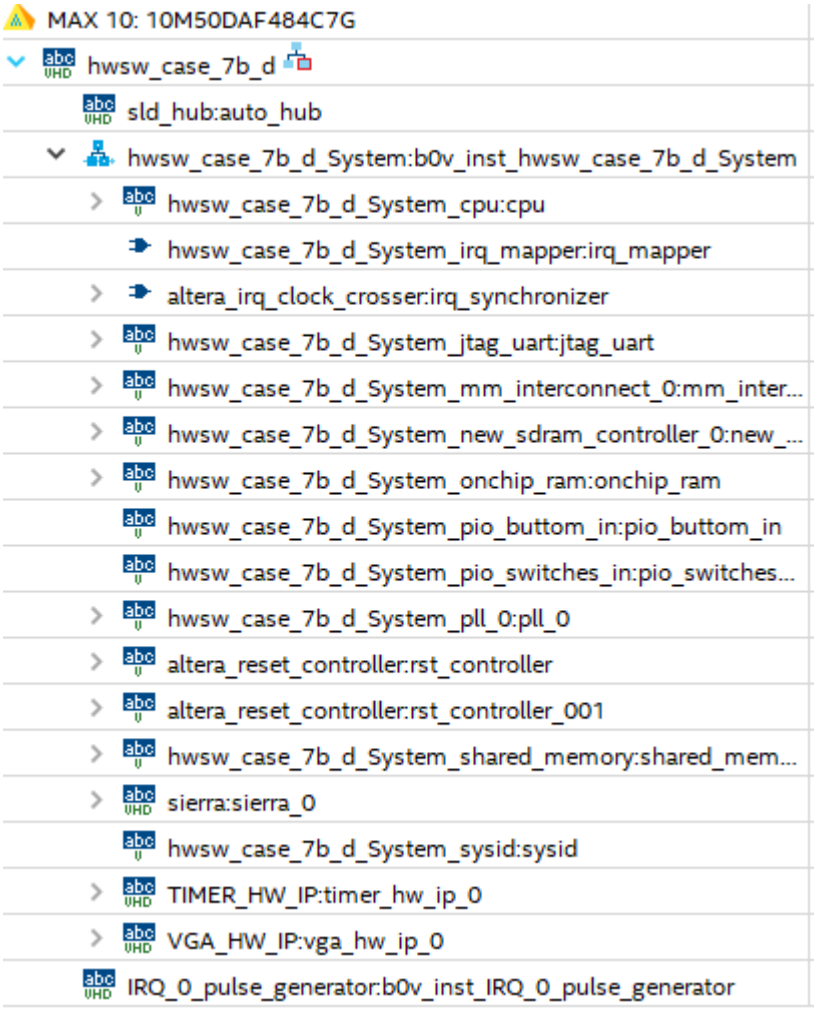
Figur 1. Systemarkitektur, blockschema för REALTIDSKÄRNOR

4.1 Systemarkitektur

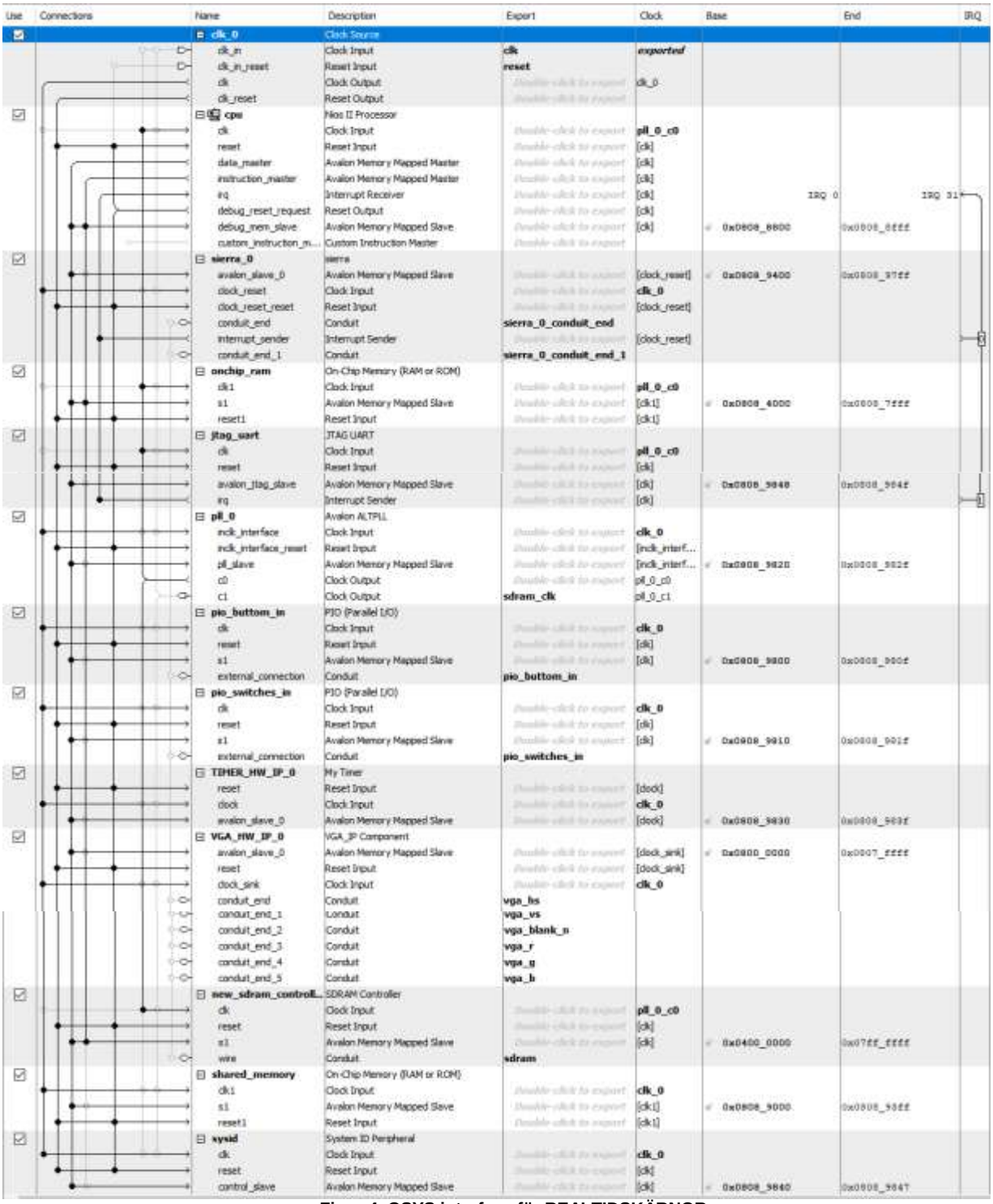
TOPPNIVÅN arkitektur visas i Figur 2, i figur 3 visar komponenthierarki och i figur 4 visar QSYS interface.



Figur 2. Systemarkitekturen för MULTI-TASKING SYSTEM



Figur 3. Komponenthierarki för REALTIDSKÄRNOR



Figur 4. QSYS interface för REALTIDSKÄRNOR

5. MJUKVARAN ARKITEKTUR

5.1 Minneskarta

Memory-map till alla de tre olika konstruktionen från ”sumary.html” för konstruktionen, se figur 5.

Nios II Memory Map

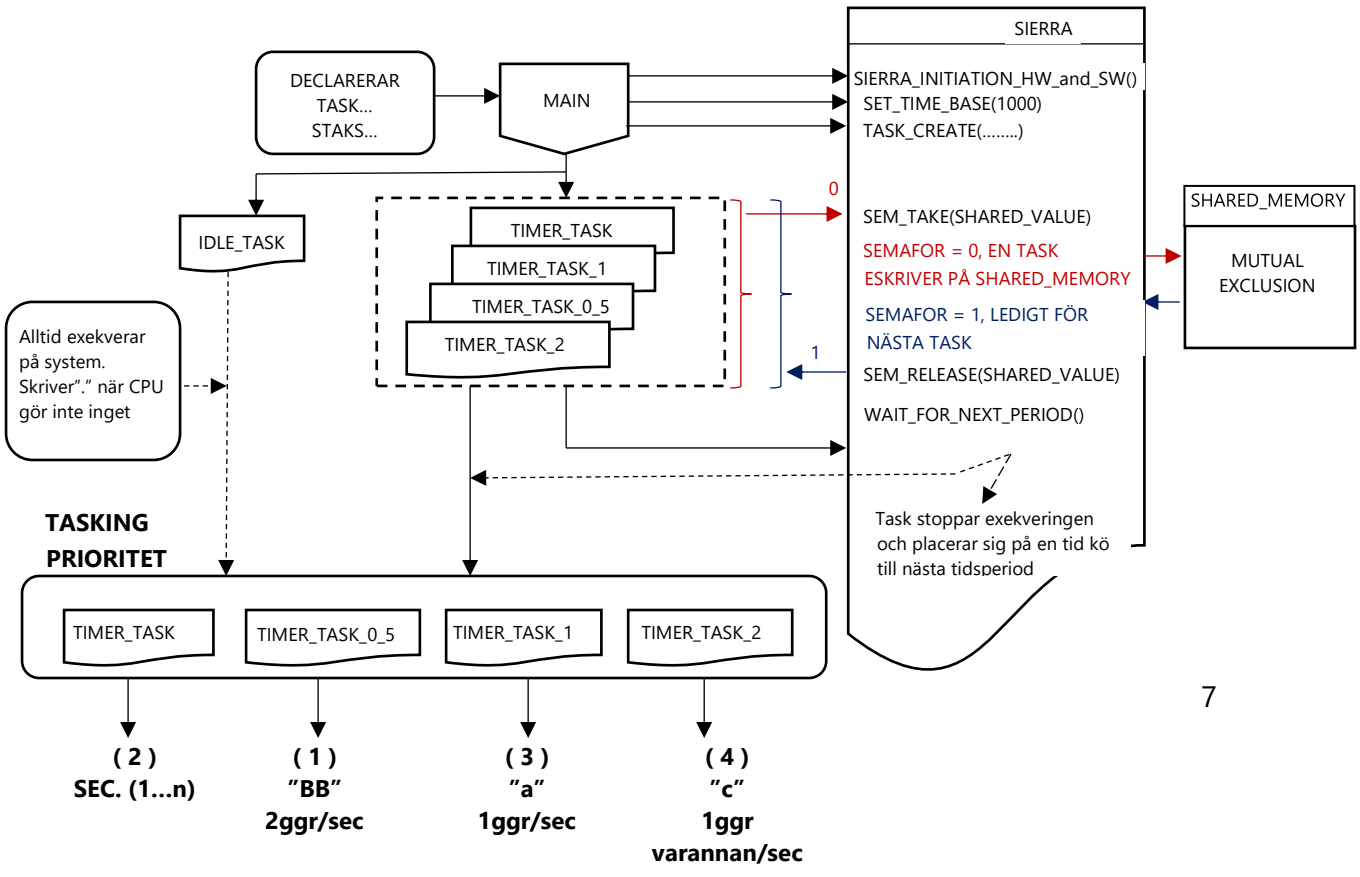
Slave Descriptor	Address Range	Size	Attributes
jtag_uart	0x08089848 - 0x0808984F	8	printable
sysid	0x08089840 - 0x08089847	8	
TIMER_HW_IP_0	0x08089830 - 0x0808983F	16	
pll_0	0x08089820 - 0x0808982F	16	
pio_switches_in	0x08089810 - 0x0808981F	16	
pio_buttom_in	0x08089800 - 0x0808980F	16	
sierra_0	0x08089400 - 0x080897FF	1024	
shared_memory	0x08089000 - 0x080893FF	1024	memory
onchip_ram	0x08084000 - 0x08087FFF	16384	memory
VGA_HW_IP_0	0x08000000 - 0x0807FFFF	524288	
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 5. Flödesdiagram REALTIDSKÄRNOR

5.2 Programarkitektur

I den här delen det kommer att se programarkitektur för alla tre olika synkroniseringsmetoder för att synkroniserar ”tasking”.

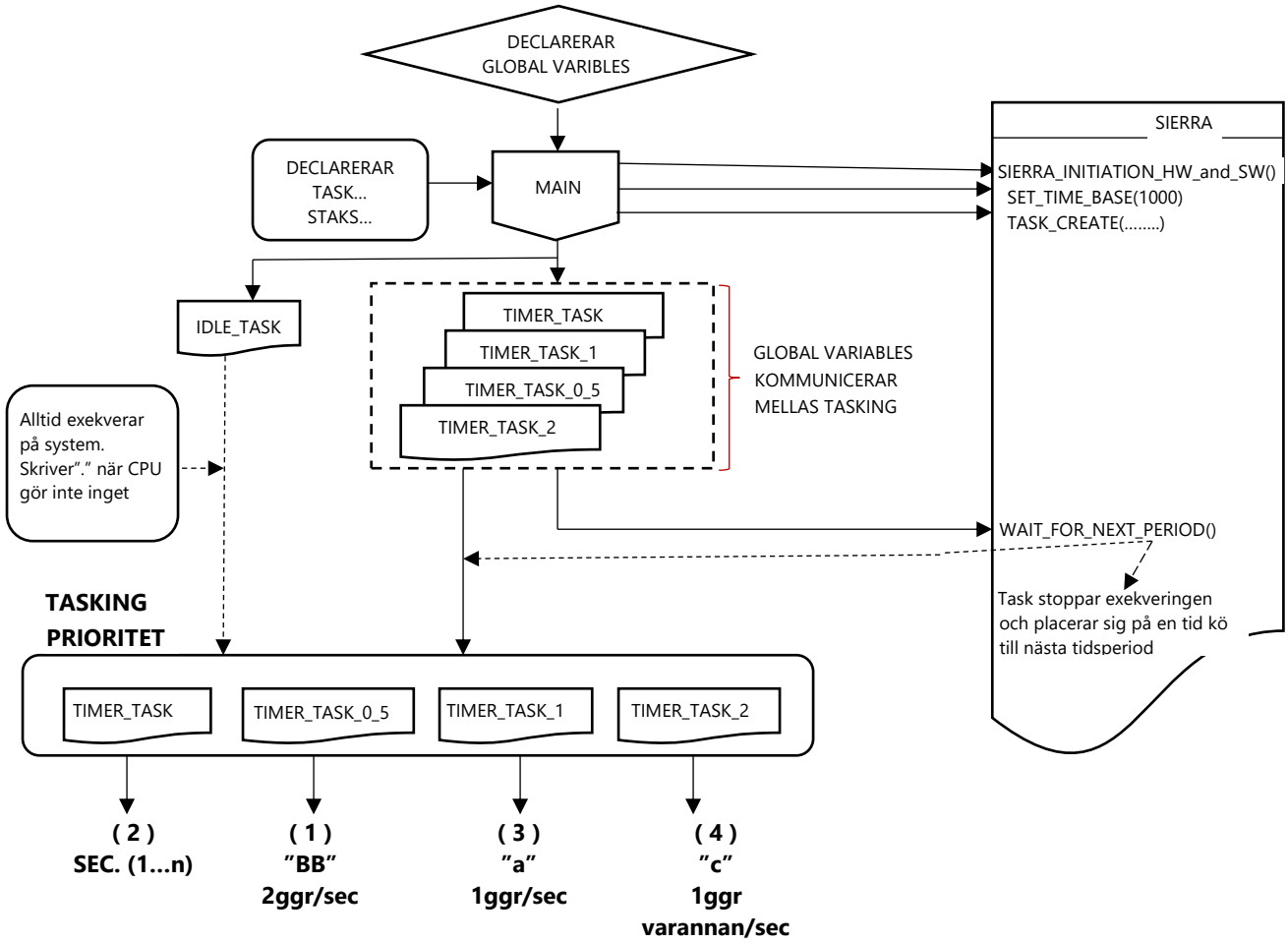
5.2.1 Semafor



Figur 6. C Arkitektur för SEMAFOR-REALTIDSKÄRNOR

I figur 6 visa programarkitektur Real-time kernel med semafor.
Alla task kan säkert skriva och läsa till det "shared memory", men en semafor måste ta innan åtkomst till det "shared minne". Den semafor får värdet "0" när är upptåget med en task. Efter att task skriver på "shared memory", den semaforen blev ledigt och returnerar värdet "1" till processen för att de andra "tasking" kan skriver på "shared memory". Detta kallas "mutual exclusion" vilket innebär att bara en task i taget kan skriva eller läsa från det "shared minne".

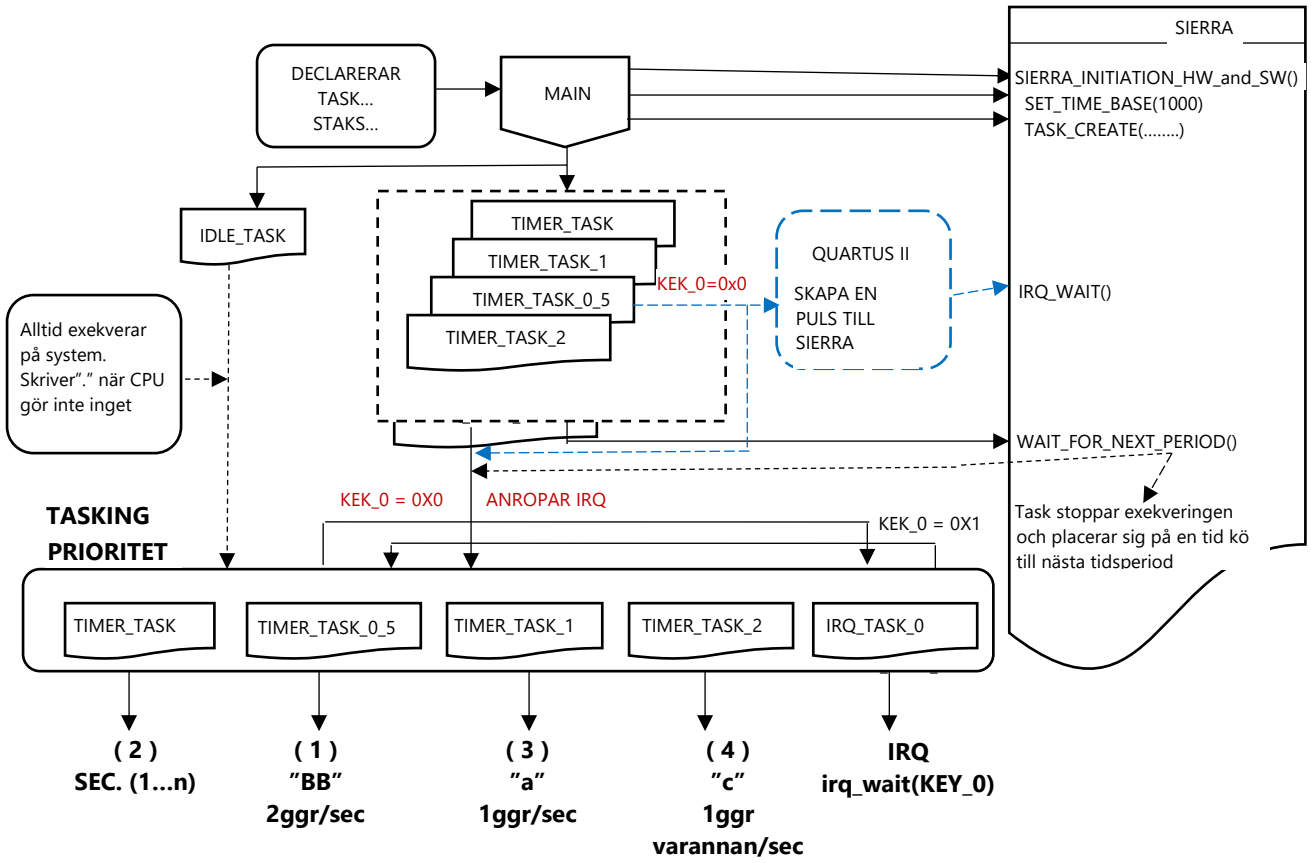
5.2.2 Global variabler



Figur 6. C Arkitektur för GLOBAL VARIABLES-REALTIDSKÄRNOR

I figur 7 visa programarkitektur Real-time kernel med global variabler.
Att deklarerar global variabler innan main är en karakteristik av den här variabler, efter att deklarerar dessa varables programmet kan använda dessa variabel för att göra att alla tasking kan kommunicerar mellan de.

5.2.2 IRQ-Sierra



Figur 7. C Arkitektur för IRQ_SIERRA-REALTIDSKÄRNOR

I figur 7 visa programarkitektur Real-time kernel med IRQ av Sierra IP komponnet.
När tryck ner KEY[0] anropar till irq_task funktionen och innuti denna funktion anropar till interfacekomponent irq_waiting och task kommer att vara i irq_waiting tillstånd. När insignalen till IRQ går högt ändras irq_task till redo. Det det finns en komponent som är byggde på hårdvaran for genererar en puls till Sierra IRQ, när KEY[0] trycks ner.

Som slutningen man kan säga att det finns många sätt att kommunicera mellan task. Kommunikationsmetoden som designern väljer beror på applikationen och de funktioner som tillhandahålls av operativsystemet.